

priority_queue 정리

가장 우선순위가 높은 값이 먼저 나오는 힙 기반 컨테이너입니다.

사용처 (Where)

- 가장 큰 값/작은 값을 반복해서 꺼낼 때
- 작업 우선순위 처리
- 다익스트라 같은 알고리즘
- 상위 K개 처리

방법 (How to Use)

- push로 삽입
- top으로 우선순위 최고 값 확인
- pop으로 제거
- 기본은 큰 값이 먼저 나옴

기본 정보

- 필요 헤더: `#include <queue>`
- 기본 선언: `priority_queue<int> pq;`

왜 써야 할까? (Why)

- 최댓값을 매번 빠르게 꺼낼 수 있음
- 직접 힙을 구현하지 않아도 됨
- 우선순위 기반 문제에서 코드가 짧아짐

주요 함수

함수/문법	의미
<code>push(x)</code>	값 삽입
<code>top()</code>	최고 우선순위 확인
<code>pop()</code>	최고 우선순위 제거
<code>empty()/size()</code>	상태 확인
<code>greater<T></code>	작은 값 우선

기본 코드 예시

```
#include <iostream>
#include <queue>
using namespace std;

int main() {
    priority_queue<int> pq;
    pq.push(10);
    pq.push(30);
    pq.push(20);

    cout << pq.top();
}
```

priority_queue 비교와 응용

STL을 직접 구현이나 C 스타일 코드와 비교하고, 실제 문제에서 쓰는 방식을 확인합니다.

시간 복잡도

동작	복잡도
push/pop	$O(\log n)$
top	$O(1)$
전체 정렬 출력	pop 반복 $O(n \log n)$

STL vs 직접 구현 vs C 스타일

구분	STL	직접 구현	C 스타일
개발 난이도	이미 구현된 기능 사용	자료구조 설계와 예외 처리 필요	배열/포인터 직접 관리
코드 길이	짧고 읽기 쉬움	길고 버그 가능성 증가	간단하지만 확장성이 낮음
성능	일반 상황에서 충분히 최적화	구현 실력에 따라 달라짐	고정 크기에서는 단순함
안정성	범위/메모리 관리 부담 감소	메모리/인덱스 실수 가능	범위 초과 위험 큼
추천 상황	대부분의 C++ 코드	자료구조 원리 학습	아주 단순한 고정 크기 데이터

응용: 작은 값부터 꺼내는 우선순위 큐

greater<int>를 사용하면 min-heap처럼 작은 값이 먼저 나옵니다.

```
#include <iostream>
#include <queue>
#include <vector>
using namespace std;

int main() {
    priority_queue<int, vector<int>, greater<int>> pq;
    pq.push(30);
    pq.push(10);
    pq.push(20);

    while (!pq.empty()) {
        cout << pq.top() << ' ';
        pq.pop();
    }
}
```

처음 배울 때 자주 하는 실수

- 전체 순회가 목적이면 적합하지 않음
- top만 우선순위가 보장되고 내부 전체가 정렬된 것은 아님
- 사용자 정의 기준은 comparator 작성 필요

비슷한 항목과 차이

- queue는 입력 순서대로 처리
- set은 정렬 상태로 전체 순회 가능
- vector+sort는 한 번 정렬 후 읽을 때 좋음

한 줄 정리: `priority_queue`은(는) '가장 큰 값/작은 값을 반복해서 꺼낼 때' 상황에서 먼저 떠올리면 좋은 STL입니다.